ЦОИ — ЛР1 — вариант 5

Поиск и обнаружение объектов методами: 1) корреляционная обработка (в пространственной области) 2) согласованная фильтрация (в частотной области)

```
clear; close all; clc;

thisDir = fileparts(mfilename("fullpath"));
imgDir = fullfile(thisDir, "Рисунки");
outDir = fullfile(thisDir, "Результаты");
if ~exist(outDir, "dir"); mkdir(outDir); end

% входные файлы
panPath = fullfile(imgDir, "Dacha.jpg");
templPaths = {
    fullfile(imgDir, "Cut_1_Dacha.jpg")
    fullfile(imgDir, "Cut_2_Dacha.jpg")
    };
templNames = ["Cut_1", "Cut_2"];

% уровни шума
noiseSigmas = [0, 0.05, 0.10, 0.15, 0.20];

% загрузка панорамы
I = im2double(imread(panPath));
if ndims(I) == 3, I = rgb2gray(I); end
I = I - mean(I(:));
I = I / max(abs(I(:)) + eps);
[mI, nI] = size(I);

% загрузка эталонов
E = cell(numel(templPaths), 1);
for k = 1:numel(templPaths)
    Ek = im2double(imread(templPaths{k}));
    if ndims(Ek) == 3, Ek = rgb2gray(Ek); end
    Ek = Ek - mean(Ek(:));
    Ek = Ek / max(abs(Ek(:)) + eps);
    E{k} = Ek;
end

% таблица результатов
rows = [];

for t = 1:numel(E)
    Et = E{t};
    [mE, nE] = size(Et);

    for s = 1:numel(noiseSigmas)
        sigma = noiseSigmas(s);
        In = I + sigma * randn(size(I));
```

```
        %- корреляционная обработка-
        tic;
        C = corrCoeffMap(In, Et);
        timeCorr = toc;
        [peakCorr, posCorr] = peak2D(C);
        bboxCorr = [posCorr(2), posCorr(1), nE, mE];

        %- согласованная фильтрация (FFT)-
        tic;
        Y = matchedFilterMapFFT(In, Et);
        timeFFT = toc;
        [peakFFT, posFFT] = peak2D(Y);
        bboxFFT = [posFFT(2) - nE + 1, posFFT(1) - mE + 1, nE, mE];

        %- визуализация карт корреляции и отклика-
        fig = figure("Color","w","Name",sprintf("%s sigma=%.2f",templNames(t),sigma));
        tiledlayout(fig, 2, 2, "Padding","compact","TileSpacing","compact");

        nexttile;
        imshow(In, []); title(sprintf("Панорама + шум \\sigma=%.2f", sigma));

        nexttile;
        imshow(Et, []); title(sprintf("Эталон %s (%dx%d)", templNames(t), mE, nE));

        nexttile;
        imshow(C, []); title(sprintf("Корреляция (spatial), peak=%.3f, t=%.3fs", peakCorr, time
        hold on; plot(posCorr(2)+nE/2, posCorr(1)+mE/2, "r+", "LineWidth", 1.5); hold off;

        nexttile;
        imshow(Y, []); title(sprintf("Соглас. фильтр (FFT), peak=%.3f, t=%.3fs", peakFFT, timeF
        hold on; plot(posFFT(2), posFFT(1), "r+", "LineWidth", 1.5); hold off;

        exportgraphics(fig, fullfile(outDir, sprintf("%s_sigma_%0.2f_maps.png", templNames(t),
        close(fig);

        %- визуализация прямоугольников на панораме-
        fig2 = figure("Color","w","Name",sprintf("%s sigma=%.2f boxes",templNames(t),sigma));
        tiledlayout(fig2, 1, 2, "Padding","compact","TileSpacing","compact");

        nexttile;
        imshow(In, []); title("Корреляция: найденный объект");
        hold on; rectangle("Position", bboxCorr, "EdgeColor","r", "LineWidth",2); hold off;

        nexttile;
        imshow(In, []); title("FFT-фильтр: найденный объект");
        hold on; rectangle("Position", bboxFFT, "EdgeColor","g", "LineWidth",2); hold off;

        exportgraphics(fig2, fullfile(outDir, sprintf("%s_sigma_%0.2f_boxes.png", templNames(t
        close(fig2);
```

```
        rows = [rows; struct( ...
            "template", templNames(t), ...
            "sigma", sigma, ...
            "corr_time_s", timeCorr, ...
            "corr_peak", peakCorr, ...
            "corr_x", bboxCorr(1), ...
            "corr_y", bboxCorr(2), ...
            "fft_time_s", timeFFT, ...
            "fft_peak", peakFFT, ...
            "fft_x", bboxFFT(1), ...
            "fft_y", bboxFFT(2) ...
            )];
    end
end

T = struct2table(rows);
writetable(T, fullfile(outDir, "results_table.csv"));
save(fullfile(outDir, "results_table.mat"), "T");
disp(T);
```

template	sigma	corr_time_s	corr_peak	corr_x	corr_y	fft_time_s	fft_peak	fft_x	fft_y
"Cut_1"	0	0.10468	0.99869	449	410	0.054427	1	449	410
"Cut_1"	0.05	0.085712	0.99381	449	410	0.053181	1	449	410
"Cut_1"	0.1	0.083036	0.98066	449	410	0.053299	1	449	410
"Cut_1"	0.15	0.08852	0.9572	449	410	0.059039	1	449	410
"Cut_1"	0.2	0.093735	0.92448	449	410	0.058745	1	449	410
"Cut_2"	0	0.11028	0.99875	811	152	0.035858	1	811	152
"Cut_2"	0.05	0.10507	0.99334	811	152	0.050195	1	811	152
"Cut_2"	0.1	0.1119	0.97733	811	152	0.040878	1	811	152
"Cut_2"	0.15	0.097517	0.9562	811	152	0.040868	1	811	152
"Cut_2"	0.2	0.11807	0.92025	811	152	0.040771	1	811	152

Локальные функции

```
function C = corrCoeffMap(I, E)
I = double(I);
E = double(E);
[mE, nE] = size(E);
win = ones(mE, nE);
nPix = mE * nE;

E0 = E - mean(E(:));
denomE = sqrt(sum(E0(:).^2) + eps);

sumI = conv2(I, win, "valid");
sumI2 = conv2(I.^2, win, "valid");
meanI = sumI / nPix;
varI = max(sumI2 - nPix .* (meanI.^2), 0);
denomI = sqrt(varI + eps);
```

```
num = conv2(I, rot90(E0, 2), "valid");
C = num ./ (denomI * denomE);
end

function Y = matchedFilterMapFFT(I, E)
I = double(I);
E = double(E);
[mI, nI] = size(I);
[mE, nE] = size(E);
m0 = mI + mE - 1;
n0 = nI + nE - 1;

E0 = E - mean(E(:));
I0 = I - mean(I(:));
E0 = E0 / max(abs(E0(:)) + eps);
I0 = I0 / max(abs(I0(:)) + eps);

HE = rot90(E0, 2);
HEp = padarray(HE, [m0 - mE, n0 - nE], "post");
Ip = padarray(I0, [m0 - mI, n0 - nI], "post");

FE = fft2(HEp);
FI = fft2(Ip);
Yc = ifft2(FE .* FI);
Y = real(Yc);
Y = Y / max(abs(Y(:)) + eps);
end

function [peak, pos] = peak2D(A)
[peakCol, rowIdx] = max(A, [], 1);
[peak, col] = max(peakCol);
row = rowIdx(col);
pos = [row, col];
end
```